

AI Boot Camp - Day 6 — Lab and Mini Project

In this chapter, we will cover the following recipes:

1. **Foundations — Greeting, Age, and Live Reactivity**
 2. **Layout and UI — Sidebar Profile, Role Access, and Dashboards**
 3. **File Upload — Reading Content and a Resume Analyzer Shell**
 4. **Session State — A Counter with Guard Rails**
 5. **Mini Project — The Simple Resume App**
-

Introduction

This is the independent lab — twelve short questions, done on your own, covering the same ground as the Activities and Mini-Challenge but without the guided setup. Each recipe below groups a few related questions together; work through them in order, since later questions in a group usually reuse the earlier one's code rather than starting fresh.

The lab ends with a **mini project** — the Simple Resume App — that combines every pattern from the four recipes into one working app: a sidebar, a file upload, a greeting, and a session-state counter. It's also a preview of Day 4's CLI Resume Analyzer becoming a real web app.

Only the Mini Project is submitted and graded. Q1–Q12 are practice — work through all of them, but nothing from those twelve questions gets handed in on their own. See *What to Submit* below for the exact file and naming convention.

What to Submit

- **Deliverable:** your working `mini_project_resume_app.py` from the *Mini Project — The Simple Resume App* recipe below. That's the only file that's submitted or graded — Q1–Q12 are practice, not deliverables.
- **Rename it before submitting:** `mini_project_<yourname>.txt` (e.g. `mini_project_janedoe.txt`) — `.txt`, not `.py`. This matches how lab files have been submitted on Moodle all bootcamp (one renamed text file per deliverable, no zip, no folder).
- **Where:** the Moodle assignment for this lab. The due date is on the Moodle assignment page itself, not repeated here.
- **Before you submit, confirm:**
 - `streamlit run mini_project_resume_app.py` starts with no errors.
 - The sidebar name field, resume upload, and the upload counter are all visible and working.
 - The file is renamed to `.txt` with your name in it — not left as `mini_project_resume_app.py`.

Foundations — Greeting, Age, and Live Reactivity

Three small variations on the same greeting app, each changing exactly one thing about how input becomes output.

Getting ready

Question	Changes from the previous one
Q1	Baseline: name in, greeting out, behind a button
Q2	Adds a second input (age) to the same greeting
Q3	Removes the button — reacts on every keystroke instead

How to do it...

1. Q1 — Greeting app:

```
import streamlit as st

st.title("Greeting App")
name = st.text_input("Enter your name")

if st.button("Greet Me"):
    st.write(f"Hello {name}")
```

2. Q2 — Add an age input to the same greeting:

```
age = st.number_input("Enter your age", min_value=0, step=1)

if st.button("Greet Me"):
    st.write(f"Hello {name}, you are {int(age)} years old")
```

3. Q3 — Make the greeting live, with no button at all:

```
if name:
    st.write(f"Hello {name}")
```

Type into the name field — the greeting appears and updates on every character, because the whole script reruns on every keystroke whether or not a button exists.

How it works...

Q1 and Q3 read the exact same widget (`st.text_input`) two different ways: gated behind `st.button` (act once, on demand) versus gated on truthiness alone (act continuously, live). Neither is "more correct" — Activity 2's sidebar in the previous chapter already used the live style for its welcome message.

See also

- *Installing Streamlit and running your first app* (doc 01) — the rerun model both styles here depend on.

Layout and UI — Sidebar Profile, Role Access, and Dashboards

Four questions building a sidebar-driven profile, a role-gated banner, and two dashboard layouts.

Getting ready

API	Role
<code>st.sidebar.selectbox</code>	A dropdown restricted to fixed choices, in the sidebar
<code>st.columns(2)</code> vs <code>st.columns([3, 1])</code>	Equal columns vs. weighted columns

How to do it...

1. Q4a — Sidebar profile:

```
import streamlit as st

st.sidebar.title("Profile")
name = st.sidebar.text_input("Name")
role = st.sidebar.text_input("Role")

st.write(f"Welcome {name}")
st.write(f"Role: {role}")
```

2. Q4b — Replace the free-text role with a restricted selectbox, and branch on it:

```
role = st.sidebar.selectbox("Role", ["Student", "Admin"])

if role == "Admin":
    st.success("Full Access Granted")
else:
    st.warning("Limited Access")
```

3. Q5 — A two-column dashboard, equal widths:

```
left, right = st.columns(2)
with left:
    st.write("Some text content here.")
with right:
    st.metric("Users", 120)
    st.metric("Status", "Active")
```

4. Q6 — The same dashboard, weighted 3:1, with an info box in the wide column:

```
left, right = st.columns([3, 1])
with left:
    st.info("This is the wide column.")
with right:
    st.write("Narrow column")
```

How it works...

Q4a's `st.sidebar.text_input("Role")` accepts *any* string — including typos and blanks. Q4b's `st.selectbox` restricts the value to a fixed set before your `if role == "Admin":` check ever runs, which is why real settings panels almost always prefer a selectbox over free text for anything a branch depends on.

See also

- *Laying out a page with a sidebar and columns* (doc 01) — `st.columns` and the sidebar API both questions build on.
- *Activity 2 — Sidebar and Columns Layout* — the guided version of this same pattern.

File Upload — Reading Content and a Resume Analyzer Shell

Three questions moving from "show the file name" to "read the content" to a named, product-shaped uploader.

How to do it...

1. Q7 — Upload a `.txt` file and show its name:

```
import streamlit as st

file = st.file_uploader("Upload a file", type=["txt"])
if file:
    st.write(f"File name: {file.name}")
```

2. Q8 — Read and display the file's content:

```
if file:
    content = file.read().decode("utf-8")
    st.write(content)
```

3. Q9 — Rename and reframe as a "Resume Analyzer" shell (UI only, no real analysis yet):

```
st.title("Resume Analyzer")
resume = st.file_uploader("Upload your resume", type=["pdf", "txt"])
if resume:
    st.success("Resume uploaded successfully!")
```

How it works...

Q7 only reads the uploaded object's `.name` attribute — the file's bytes are never touched. Q8 calls `.read().decode("utf-8")`, which only works because `.txt` is plain text; the same call on a `.pdf` would produce binary garbage, which is why Q9's Resume Analyzer stops at "uploaded successfully" instead of trying to read a PDF's content the same way.

See also

- *Uploading and reading files* (doc 01) — the `.read().decode(...)` gotcha for PDFs, spelled out in full.

Session State — A Counter with Guard Rails

Three questions building the same counter as Activity 4, then adding a floor so it can't go negative.

How to do it...

1. Q10 — Basic counter:

```
import streamlit as st

if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Add +1"):
    st.session_state.count += 1

st.write(st.session_state.count)
```

2. Q11 — Add a reset button:

```
if st.button("Reset"):
    st.session_state.count = 0
```

3. Q12 (Challenge) — Add a subtract button that never goes below zero:

```
if st.button("Subtract -1"):
    if st.session_state.count > 0:
        st.session_state.count -= 1
```

How it works...

The guard in Q12 — `if st.session_state.count > 0:` — has to sit *inside* the button's `if`, not around the whole app: it needs to check the count fresh on the one rerun where "Subtract" was actually clicked, not on every rerun regardless of which button (if any) was pressed.

See also

- *Persisting data across reruns with session_state* (doc 02) — the init-guard pattern every question in this recipe depends on.
 - *Activity 4 — The Persistent Counter* — the guided version this lab recipe extends with a floor.
-

Mini Project — The Simple Resume App

Every pattern from this chapter, combined into one small app: a sidebar identity, a file upload, a greeting built from both, and a session-state counter tracking how many times a file has been uploaded this session.

Getting ready

Piece	Comes from
Sidebar name input	Layout and UI recipe
Resume upload	File Upload recipe
Upload counter	Session State recipe

How to do it...

1. Create `mini_project_resume_app.py`:

```
import streamlit as st

st.sidebar.title("Your Info")
name = st.sidebar.text_input("Name")

st.title("Simple Resume App")

if "upload_count" not in st.session_state:
    st.session_state.upload_count = 0
if "last_file_id" not in st.session_state:
    st.session_state.last_file_id = None

resume = st.file_uploader("upload your resume", type=["pdf", "txt"])
if resume:
    if resume.file_id != st.session_state.last_file_id:
        st.session_state.upload_count += 1
        st.session_state.last_file_id = resume.file_id
    st.success(f"Hello {name}, we received: {resume.name}")

st.metric("Times uploaded this session", st.session_state.upload_count)
```

2. **Upload the same file three times in a row** (remove it with the "x" and re-browse each time — that's what actually creates a new upload event). The metric climbs to 3. Then, with a file still uploaded, type something unrelated in the sidebar "Name" field — the metric does **not** move, even though that keystroke triggers a rerun and `resume` is still truthy.

How it works...

Nothing in this file is new: it's Q4a's sidebar input, Q9's resume uploader, and Q10's session-state counter, assembled into one script — but assembling them exposes a bug none of the three showed on its own.

Gotcha: `st.file_uploader` is **not** a one-shot widget like `st.button` or `st.chat_input`. Once a file is uploaded, `resume` stays truthy on *every* rerun after that — including a rerun caused by typing in the unrelated "Name" field — until the file is removed. `if resume: st.session_state.upload_count += 1` with no other guard therefore increments on every one of those unrelated reruns too, not just on a genuine new upload — the exact "code outside the real gate" bug doc 02's *Debugging the classic session-state bugs* recipe already catalogs, just wearing a file-uploader costume instead of a chat one. The fix follows that recipe's own medicine: gate the increment on something that only changes on a real new upload — `resume.file_id`, a value `UploadedFile` sets fresh per upload — instead of on `resume`'s bare truthiness.

There's more...

This is the same shell Day 7 turns into a real backend: swap the local `st.file_uploader` handling for a call to Day 4's CLI Resume Analyzer logic (or, following this chapter's own arc, a Flask endpoint that does the analysis server-side — see doc 04 onward).

See also

- *Uploading and reading files* (doc 01) — the file-upload half of this mini project.
- *Persisting data across reruns with session_state* (doc 02) — the counter half.
- *Building a stateful CRUD API for scene objects* (doc 04) — what this same "upload count" idea looks like once it lives on a server instead of in one browser session.